

05. Python Foundations

Detailed beginner handbook - English and Arabic

Technologies: Python

What you will learn

Filter synthetic records and calculate a result.

Python uses readable syntax and is well suited to APIs, automation, spreadsheets, and data work.

الهدف: بناء مثال صغير وفهمه خطوة بخطوة.

شرح بسيط: تعلم الفكرة أولاً، ثم جرب المثال الصغير ببيانات اصطناعية فقط.

How to study

Study one lesson at a time. Type examples yourself, predict the result, run the code, then change one thing. Keep a notebook of new words, errors, root causes, and fixes.

Course map

Setup and mental model; ten core concepts; a guided mini-project; the real dashboard architecture; debugging; exercises and answers.

Lesson 1 - Setup and mental model

Prerequisites

Windows 10 or 11, VS Code, PowerShell, and permission to install learning dependencies. Use only synthetic data in tutorials and screenshots.

Setup sequence

1. Open PowerShell.
2. Go to `learning-examples/05-python-foundations`.
3. Read `README.md`.
4. Run `python app.py`.
5. Read the complete error if it fails.
6. Change one safe value and rerun.

الخطوات: افتح مجلد المثال، شغل الأمر، غير قيمة واحدة، ولاحظ النتيجة.

Mental model

Treat Python as one layer in a system. Identify the input, the transformation or rule, the output, and possible failures. Understanding that flow is more valuable than memorising syntax.

Checkpoint

Explain: What problem does this technology solve? What is its input? What output should it produce? What can fail?

Lesson 2 - Running Python, indentation, comments, and expressions

What is it?

Python executes statements in order. Indentation defines blocks. Comments begin with #. An expression produces a value; a statement performs an action such as assignment.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Python Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
# hello.py
print('Hello, Python!')
# Run: python hello.py
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Running Python, indentation, comments, and expressions.

Running Python, indentation, comments, and expressions: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 3 - Numbers, strings, booleans, None, and type conversion

What is it?

int and float store numbers; str stores text; bool stores truth values; None represents absence. Explicit conversion such as int(text) can fail and should be validated.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Python Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
total = 12          # int
rate = 0.75         # float
month = 'January'   # str
ready = True        # bool
missing = None
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Numbers, strings, booleans, None, and type conversion.

Numbers, strings, booleans, None, and type conversion: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 4 - Lists, tuples, dictionaries, sets, and comprehensions

What is it?

Lists are ordered and mutable, tuples are fixed sequences, dictionaries map keys to values, and sets keep unique values. Comprehensions build collections concisely.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Python Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
months = ['Jan', 'Feb']
point = (10, 20)
summary = {'month': 'Jan', 'total': 12}
unique_ids = {'A1', 'A2'}
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Lists, tuples, dictionaries, sets, and comprehensions.

Lists, tuples, dictionaries, sets, and comprehensions: اكتب المثال بنفسك، حدد المدخلات والعمليات والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 5 - Conditions, for loops, while loops, and control flow

What is it?

if chooses a branch, for iterates items, while repeats while a condition remains true, break exits, and continue skips to the next iteration.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Python Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
for total in [4,8,0]:
    if total > 0:
        print('Has data')
    else:
        print('Empty')
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Conditions, for loops, while loops, and control flow.

Conditions, for loops, while loops, and control flow: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 6 - Functions, parameters, return values, and scope

What is it?

Functions name reusable work. Parameters receive input; return sends output. Default and keyword arguments improve clarity. Local variables normally exist only inside their function.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Python Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
def percentage(part, whole):  
    if whole == 0:  
        return 0  
    return part / whole * 100  
print(percentage(3, 12))
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Functions, parameters, return values, and scope.

Functions, parameters, return values, and scope: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Modules, imports, packages, and virtual environments

What is it?

A module is a Python file; a package groups modules. Imports reuse code. A virtual environment isolates project dependencies from other applications.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Python Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
# utils.py: def add(a,b): return a+b
from pathlib import Path
print(Path.cwd())
# Create isolation: python -m venv .venv
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Modules, imports, packages, and virtual environments.

Modules, imports, packages, and virtual environments: اكتب المثال بنفسك، حدد المدخلات والعمليات والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 8 - Files, paths, context managers, JSON, and CSV

What is it?

Pathlib builds paths safely. with closes files automatically. json handles structured text; csv handles rows and columns. Always specify encoding when reading text.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Python Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
from pathlib import Path
import json
data = {'total':12}
Path('summary.json').write_text(json.dumps(data), encoding='utf-8')
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Files, paths, context managers, JSON, and CSV.

Files, paths, context managers, JSON, and CSV: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 9 - Exceptions, validation, logging, and useful error messages

What is it?

Raise exceptions when a function cannot fulfil its contract. Catch only errors you can handle, add context, and log operational facts without leaking sensitive data.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Python Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
try:
    total = int(input('Total: '))
except ValueError as exc:
    print('Enter a whole number')
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Exceptions, validation, logging, and useful error messages.

Exceptions, validation, logging, and useful error messages: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 10 - Classes, dataclasses, and when objects are useful

What is it?

A class combines state and behaviour; a dataclass reduces boilerplate for data records. Prefer functions and dictionaries until an object clearly improves the design.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Python Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
from dataclasses import dataclass
@dataclass
class Summary:
    month: str
    total: int
print(Summary('Jan', 12))
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Classes, dataclasses, and when objects are useful.

Classes, dataclasses, and when objects are useful: هذا الدرس يشرح: حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 11 - Testing, debugging, style, and safe handling of sensitive data

What is it?

Small functions, type hints, tests, a debugger, and readable formatting support change. Validate files before processing and minimise personal data throughout the pipeline.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Python Foundations, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
def total(values: list[int]) -> int:
    return sum(values)

def test_total():
    assert total([4,8]) == 12
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Testing, debugging, style, and safe handling of sensitive data.

Testing, debugging, style, and safe handling of sensitive data: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Guided mini-project

Project goal

Filter synthetic records and calculate a result.

Starting example

```
records=[{'status':'open'},{'status':'closed'}]
def open_count(rows):
    return sum(row['status']=='open' for row in rows)
print(open_count(records))
```

Read the code

Find imported tools or page elements. Identify the synthetic input. Find the function, event, route, or transformation that changes it. Finally, locate where the result is displayed, returned, saved, or packaged.

Build sequence

1. Run the untouched example.
2. Record the result.
3. Rename one unclear value.
4. Add a synthetic record.
5. Validate empty input.
6. Validate incorrect input.
7. Add a helpful error.
8. Rerun every case.
9. Compare with exercise-solution.
10. Explain every line.

Definition of done

Normal, empty, and invalid cases behave deliberately; no private data is used; and you can explain every line without reading the guide.

Lesson 8 - The real dashboard

Architecture connection

The application combines Python with the rest of the stack. The frontend gathers filters and files; the local API validates requests; the data layer transforms workbook rows; charts present aggregates; export tools create files; and the desktop layer packages the application for Windows.

Trace the upload feature

The user selects a workbook. The frontend sends it to the local API. The backend validates the file and sheets. The data layer creates safe tables. The API returns metadata. React updates state. Plotly displays results. Identify exactly where this guide's technology participates.

Privacy and quality

Do not place narrative, contact, or identifying fields in tutorials, logs, screenshots, tests, or exports. Use generated identifiers and synthetic totals. Validate at each boundary and collect only fields required for the calculation.

Architecture exercise

Draw five boxes: UI, API, Data Processing, Export, Desktop Packaging. Add arrows and label the data. Circle the box related to this guide and add two validation checks.

Lesson 9 - Debugging

Reliable debugging loop

1. Reproduce with the smallest input.
2. Read the first meaningful error.
3. Find the exact file and line.
4. Inspect value and type.
5. Compare expected and actual results.
6. Change one thing.
7. Rerun the same case.
8. Add a regression test.

Common beginner mistakes

- Wrong folder or command.
- Missing dependency or inactive environment.
- Misspelled file, variable, field, route, or import.
- Assuming input is always present.
- Mixing setup, processing, and display in one block.
- Hiding an exception rather than understanding it.
- Using real sensitive data in a test.

أخطاء شائعة: تشغيل الأمر من مجلد خاطئ، نسيان تثبيت التبعيات، أو تغيير أشياء كثيرة بمرة واحدة.

Error notebook

Record: command, expected result, actual error, root cause, smallest fix, and test added. This turns errors into reusable knowledge.

Lesson 10 - Exercises and answers

Exercises

1. Explain the technology in three sentences.
2. Recreate the example without copying.
3. Add one normal synthetic record.
4. Handle empty input.
5. Handle invalid input.
6. Improve unclear names.
7. Add or describe one test.
8. Locate the technology in the dashboard.
9. Record one error and root cause.
10. Add one mini-project feature.

تمرين: أضف شهرا أو قيمة أمانة جديدة، ثم اشرح ما حدث بكلماتك.

Answer guide

A strong solution is observable and explainable: the documented command works; output changes for the new record; empty and invalid cases have deliberate results; names communicate purpose; a test states an expected result; and the architecture explanation identifies the correct boundary.

Next step

Next: 06. FastAPI and Uvicorn